

```
!pip install transformers torch
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.8.0+cu126)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.27.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.20.3)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.6.2)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transformers) (1.1.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.37.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.31.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025.11.11)
```

✶ Bài 1: Khởi phục Masked Token (Masked Language Modeling)

Code:

```
from transformers import pipeline
# 1. Tải pipeline "fill-mask"
# Pipeline này sẽ tự động tải một mô hình mặc định phù hợp (thường là một biến thể của BERT)
mask_filler = pipeline("fill-mask")
# 2. Câu đầu vào với token <mask>
input_sentence = "Hanoi is the <mask> of Vietnam."
# 3. Thực hiện dự đoán
# top_k=5 yêu cầu mô hình trả về 5 dự đoán hàng đầu
predictions = mask_filler(input_sentence, top_k=5)
# 4. In kết quả
print(f"Câu gốc: {input_sentence}")
for pred in predictions:
    print(f"Dự đoán: '{pred['token_str']}' với độ tin cậy: {pred['score']:.4f}")
    print(f"-> Câu hoàn chỉnh: {pred['sequence']}")
```

```
No model was supplied, defaulted to distilbert/distilroberta-base and revision fb53ab8 (https://huggingface.co/distilbert/di)
Using a pipeline without specifying a model name and revision in production is not recommended.
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100%                               480/480 [00:00<00:00, 27.6kB/s]

model.safetensors: 100%                         331M/331M [00:02<00:00, 240MB/s]

Some weights of the model checkpoint at distilbert/distilroberta-base were not used when initializing RobertaForMaskedLM: [
- This IS expected if you are initializing RobertaForMaskedLM from the checkpoint of a model trained on another task or with
- This IS NOT expected if you are initializing RobertaForMaskedLM from the checkpoint of a model that you expect to be exact

tokenizer_config.json: 100%                      25.0/25.0 [00:00<00:00, 3.10kB/s]

vocab.json:      899k/? [00:00<00:00, 23.7MB/s]

merges.txt:      456k/? [00:00<00:00, 29.0MB/s]

tokenizer.json:   1.36M/? [00:00<00:00, 48.6MB/s]

Device set to use cuda:0
Câu gốc: Hanoi is the <mask> of Vietnam.
Dự đoán: ' capital' với độ tin cậy: 0.9341
-> Câu hoàn chỉnh: Hanoi is the capital of Vietnam.
Dự đoán: ' Republic' với độ tin cậy: 0.0300
-> Câu hoàn chỉnh: Hanoi is the Republic of Vietnam.
Dự đoán: ' Capital' với độ tin cậy: 0.0105
-> Câu hoàn chỉnh: Hanoi is the Capital of Vietnam.
Dự đoán: ' birthplace' với độ tin cậy: 0.0054
-> Câu hoàn chỉnh: Hanoi is the birthplace of Vietnam.
Dự đoán: ' heart' với độ tin cậy: 0.0014
-> Câu hoàn chỉnh: Hanoi is the heart of Vietnam.
```

Trả lời câu hỏi:

1. Mô hình đã dự đoán đúng từ "capital" không?

Mô hình đã dự đoán đúng từ `capital` với độ tin cậy rất cao: **0.9341**.

2. Tại sao các mô hình Encoder-only như BERT lại phù hợp cho tác vụ này?

Kết quả `Hanoi is the <mask> of Vietnam` → `capital` là một ví dụ sách giáo khoa để giải thích sức mạnh vượt trội của kiến trúc **Encoder-only**.

A. Hiểu ngữ cảnh hai chiều (Bidirectional Context)

Khác biệt lớn nhất so với các mô hình RNN/LSTM truyền thống là khả năng xử lý ngữ cảnh:

- **Hạn chế của mô hình đơn hướng:** Để điền đúng từ vào `<mask>`, mô hình không thể chỉ đọc từ trái sang phải (`Hanoi is the...`). Nếu chỉ đọc chiều này, mô hình có thể điền là `"city"`, `"pride"`, `"largest city"`, v.v.
- **Sức mạnh của Encoder:** Mô hình nhìn thấy **cả hai phía cùng lúc**: Nó thấy chủ ngữ là **"Hanoi"** VÀ bổ ngữ phía sau là **"of Vietnam"**.
- **Kết quả:** Sự kết hợp giữa "Hanoi" và "Vietnam" tạo ra một mối liên kết ngữ nghĩa mạnh mẽ nhất là quan hệ thủ đô - đất nước. Chính việc **"nhìn thấy tương lai"** (từ "Vietnam") đã giúp nó loại bỏ các từ sai và chọn đúng từ `capital`.

B. Masked Language Modeling (MLM)

Mô hình đã được huấn luyện chuyên biệt để làm nhiệm vụ này:

- **Mục tiêu huấn luyện:** Mô hình đã được huấn luyện bằng cách che đi (mask) hàng tỷ từ trong văn bản và buộc phải **đoán lại chúng** dựa trên ngữ cảnh xung quanh.
- **Tính phù hợp:** Bài toán điền từ vào chỗ trống là *sở trường* gốc của mô hình, giúp nó nhạy bén hơn bất kỳ mô hình RNN nào.
- **Lưu ý kỹ thuật:** Việc bạn thấy ký hiệu `<mask>` (thay vì `[MASK]`) và các từ có khoảng trắng phía trước (ví dụ `' capital'`) cho thấy bạn có thể đang sử dụng mô hình **RoBERTa** (một biến thể được tối ưu hóa hơn của BERT).

✦ Bài 2: Dự đoán từ tiếp theo (Next Token Prediction)

Code:

```
from transformers import pipeline
# 1. Tải pipeline "text-generation"
```

```
# Pipeline này sẽ tự động tải một mô hình phù hợp (thường là GPT-2)
generator = pipeline("text-generation")
# 2. Đoạn văn bản mẫu
prompt = "The best thing about learning NLP is"
# 3. Sinh văn bản
# max_length: tổng độ dài của câu mẫu và phần được sinh ra
# num_return_sequences: số lượng chuỗi kết quả muốn nhận
generated_texts = generator(prompt, max_length=50, num_return_sequences=1)
# 4. In kết quả
print(f"Câu mẫu: '{prompt}'")
for text in generated_texts:
    print("Văn bản được sinh ra:")
    print(text['generated_text'])
```

community/gpt2 and revision 607a30d (<https://huggingface.co/openai-community/gpt2>).
name and revision in production is not recommended.

`max\_length` is provided a specific value, please use `truncation=True` to explicitly truncate examples to max length. Default truncation length is 512 for open-end generation.
If `max\_length` is not provided, the maximum length of the output is determined by the maximum length of the input and the maximum length of the output. If `max\_length` is provided, it will take precedence. Please refer to the documentation for more information.

is 'The best thing about learning NLP is that it is an academic discipline in which the faculty members are very smart and creative, and the students are extremely interested in learning to teach the NLP course without some work. You're going to learn all the techniques that I would have worked on in my first year of thought of. The problem with being a NLP student is that you're just not going to get the benefit of the NLP. How do you get it? Well, there are a lot of different ways to get back into the NLP. If you're going to study in the NLP, then you have to go back to the beginning and start from the beginning.

Trả lời câu hỏi:

1. Kết quả sinh ra có hợp lý không?

Đoạn văn bản được sinh ra không hợp lý về mặt ngữ nghĩa và logic. Tuy có ngữ pháp đúng nhưng mắc các lỗi nghiêm trọng về cấu trúc và ý nghĩa (điển hình của các mô hình RNN/LSTM không được tinh chỉnh hoặc chiến thuật sinh văn bản tham lam):

Vấn đề	Phân tích chi tiết
Lặp lại (Repetition)	Mô hình bị mắc kẹt trong việc lặp lại các cụm từ hoặc ý tưởng gần kề. Ví dụ: <i>"The NLP faculty are very smart and creative."</i> (lặp lại ý câu trước) và lặp lại vì
Mâu thuẫn logic	Mô hình tự mâu thuẫn trong lập luận. Ví dụ: Dù câu mẫu bắt đầu bằng ý tích cực (<i>"The best thing..."</i>), mô hình lại sinh ra: <i>"The problem with being a NLP student is that you're just not going to get the benefit of the NLP."</i>
Trôi chủ đề (Topic Drift)	Mô hình mất khả năng duy trì ngữ cảnh dài. Nó đang nói về việc học NLP, sau đó đột ngột chuyển sang các ngành khác như <i>"law school"</i> (trường luật) mà

2. Tại sao các mô hình Decoder-only như GPT lại phù hợp cho tác vụ này?

Tác vụ này là **Sinh văn bản tự do (Open-ended Text Generation)**, và các mô hình Decoder-only (như GPT, GPT-2, GPT-3) là kiến trúc tối ưu nhất nhờ vào đặc tính **tự hồi quy** và kiến trúc **Transformer**:

A. Sinh văn bản Tự hồi quy (Autoregressive Generation)

- Mô hình Decoder được thiết kế để dự đoán token tiếp theo (token) dựa trên tất cả các token đã được sinh ra trước đó.
- Quá trình này lặp đi lặp lại cho đến khi sinh ra đủ độ dài hoặc gặp token kết thúc chuỗi (<EOS>). Đây là cơ chế cơ bản để tạo ra văn bản tuần tự và mạch lạc.

B. Cơ chế Masked Self-Attention

- Trong kiến trúc Decoder, cơ chế **Self-Attention** được "che mặt" (Masked). Điều này đảm bảo rằng khi mô hình đang dự đoán một từ, nó **chỉ nhìn vào các từ đã xuất hiện** (phía bên trái), chứ không nhìn vào các từ "tương lai" chưa được sinh ra.
- Đây là điều kiện bắt buộc và lý tưởng cho tác vụ sinh văn bản (chúng ta không thể biết trước kết quả).

C. Ngữ cảnh dài và Tính mạch lạc

- Nhờ sử dụng kiến trúc **Transformer**, mô hình Decoder-only có thể xử lý và duy trì thông tin ngữ cảnh qua **hàng ngàn token** (độ dài cửa sổ ngữ cảnh), khắc phục hoàn toàn vấn đề **Trôi chủ đề** và **Lặp lại** của các mô hình RNN/LSTM khi sinh văn bản dài.
- Việc huấn luyện trên dữ liệu khổng lồ (Pre-training) giúp GPT học được logic, cấu trúc tường thuật, và cách đặt câu hỏi/trả lời một cách tự nhiên và có ý nghĩa.

✦ Bài 3: Tính toán Vector biểu diễn của câu (Sentence Representation)

Code:

```
import torch
from transformers import AutoTokenizer, AutoModel
```

```
# 1. Chọn một mô hình BERT
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
# 2. Câu đầu vào
sentences = ["This is a sample sentence."]
# 3. Tokenize câu
# padding=True: đệm các câu ngắn hơn để có cùng độ dài
# truncation=True: cắt các câu dài hơn
# return_tensors='pt': trả về kết quả dưới dạng PyTorch tensors
inputs = tokenizer(sentences, padding=True, truncation=True, return_tensors='pt')
# 4. Đưa qua mô hình để lấy hidden states
# torch.no_grad() để không tính toán gradient, tiết kiệm bộ nhớ
with torch.no_grad():
    outputs = model(**inputs)
# outputs.last_hidden_state chứa vector đầu ra của tất cả các token
last_hidden_state = outputs.last_hidden_state
# shape: (batch_size, sequence_length, hidden_size)
# 5. Thực hiện Mean Pooling
# Để tính trung bình chính xác, chúng ta cần bỏ qua các token đệm (padding tokens)
attention_mask = inputs['attention_mask']
mask_expanded = attention_mask.unsqueeze(-1).expand(last_hidden_state.size()).float()
sum_embeddings = torch.sum(last_hidden_state * mask_expanded, 1)
sum_mask = torch.clamp(mask_expanded.sum(1), min=1e-9)
sentence_embedding = sum_embeddings / sum_mask
# 6. In kết quả
print("Vector biểu diễn của câu:")
print(sentence_embedding)
print("\nKích thước của vector:", sentence_embedding.shape)
```



```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.00kB/s]
config.json: 100% 570/570 [00:00<00:00, 73.6kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 3.57MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 7.36MB/s]
model.safetensors: 100% 440M/440M [00:04<00:00, 111MB/s]
```

Vector biểu diễn của câu:

```
tensor([[ -6.3875e-02, -4.2837e-01, -6.6779e-02, -3.8430e-01, -6.5785e-02,
         -2.1826e-01,  4.7636e-01,  4.8659e-01,  3.9689e-05, -7.4274e-02,
        -7.4740e-02, -4.7635e-01, -1.9773e-01,  2.4824e-01, -1.2162e-01,
         1.6678e-01,  2.1045e-01, -1.4576e-01,  1.2637e-01,  1.8635e-02,
         2.4640e-01,  5.7090e-01, -4.7014e-01,  1.3782e-01,  7.3650e-01,
        -3.3808e-01, -5.0329e-02, -1.6453e-01, -4.3517e-01, -1.2900e-01,
         1.6516e-01,  3.4004e-01, -1.4930e-01,  2.2422e-02, -1.0488e-01,
        -5.1916e-01,  3.2964e-01, -2.2162e-01, -3.4206e-01,  1.1993e-01,
        -7.0148e-01, -2.3126e-01,  1.1224e-01,  1.2550e-01, -2.5191e-01,
        -4.6374e-01, -2.7261e-02, -2.8415e-01, -9.9250e-02,  3.7018e-02,
        -8.9192e-01,  2.5005e-01,  1.5816e-01,  2.2701e-01, -2.8497e-01,
         4.5300e-01,  5.0921e-03, -7.9441e-01, -3.1008e-01, -1.7403e-01,
         4.3029e-01,  1.6816e-01,  1.0590e-01, -4.8987e-01,  3.1856e-01,
         2.2861e-01, -1.3403e-02,  1.8807e-01, -1.0905e+00,  2.1010e-01,
        -6.7579e-01, -5.7076e-01,  8.5946e-02,  1.9121e-01, -3.3818e-01,
         2.7744e-01, -4.0539e-01,  3.1305e-01, -4.1197e-01, -5.6820e-01,
        -3.9074e-01,  4.0747e-01,  9.9898e-02,  2.3719e-01,  1.0154e-01,
        -2.5670e-01, -2.0583e-01,  1.1763e-01, -5.1439e-01,  4.0979e-01,
         1.2149e-01,  1.9333e-02, -5.9029e-02, -2.0141e-01,  7.0860e-01,
        -4.9832e-02,  2.4780e-02, -9.0585e-03,  1.9667e-02,  3.0815e-01,
        -4.9832e-02, -1.0691e+00,  6.1072e-01, -4.9723e-02, -1.5156e-01,
        -6.7778e-02,  4.7811e-02,  5.2102e-01,  1.6951e-01,  1.0145e-02,
         5.3993e-01,  7.8189e-02, -6.9842e-02,  3.9833e-01,  6.0460e-01,
         4.2071e-01,  1.1822e-01,  2.3631e-01, -4.5379e-02, -1.3740e-01,
        -4.4018e-01, -6.8122e-02,  1.9934e-01,  8.7062e-01, -2.2603e-01,
        -3.6010e-01,  2.0226e-01,  3.7898e-01,  1.9533e-01, -3.0366e-01,
         3.8633e-01,  6.1949e-01,  6.8663e-01, -1.8068e-01, -3.6815e-01,
         2.8512e-01,  1.0890e-01,  2.5938e-01, -4.2975e-01,  1.3345e-01,
        -1.0252e-01, -8.9736e-02,  4.7384e-01,  8.1717e-02,  1.5885e-01,
         4.0251e-01,  1.2887e-01, -3.7689e-01, -3.4447e-01, -4.2116e-01,
        -1.0252e-01, -8.9736e-02,  4.7384e-01,  8.1717e-02,  1.5885e-01,
        -3.8330e-02, -2.0035e-01,  2.6654e-01,  9.3773e-02, -4.6780e-02,
        -4.0519e-01, -4.4705e-01,  8.9500e-01, -3.8333e-01,
        -2.0583e-02,  1.5505e-01,  3.9070e-01,  3.9616e-01,
        -2.0599e+00,  6.7052e-01,  2.1112e-01, -4.7306e-01,  3.4865e-01,
        -2.0319e-01, -7.3276e-02, -3.8147e-01, -5.4455e-01,  3.5050e-01,
        2.249e-01, -2.1471e-01, -3.8439e-01, -1.0760e-01, -8.8821e-02,
        2.5263e-01,  2.1448e-01,  5.5798e-02, -6.5411e-02,  9.9838e-02,
        -5.5214e-01,  1.1125e+00,  5.4141e-01, -7.4160e-02,  3.5337e-01,
        1.2513e-01,  3.4856e-02,  2.0560e-01, -1.2517e-01, -4.4332e-02,
        -2.9441e-01, -1.4139e-01,  1.1662e-01, -3.6223e-01, -1.4621e-01,
        6.5255e-02,  3.9270e-01,  3.8543e-01, -2.3996e-01, -3.1482e-01,
        -3.9838e-01, -3.6006e-01, -1.9672e-01, -2.7738e-01, -4.1097e-01,
        3.6456e-01, -2.6012e-01,  1.2587e-01,  1.2752e-01,  5.4261e-01,
        1.0569e-01,  3.5704e-01,  1.4766e-01,  4.4929e-01, -8.1255e-01,
        -3.0410e-02,  5.8064e-02,  2.0699e-01,  6.6129e-01,  3.9243e-01,
        -6.8644e-01, -8.3415e-01, -1.2653e-01,  1.9644e-01, -4.0900e-01,
        -6.3775e-02, -1.8780e-01,  7.9474e-02, -1.7443e-01,  3.1936e-01,
```

Trả lời câu hỏi

1. Kích thước (chiều) của vector biểu diễn là bao nhiêu? Tương ứng tham số nào?

- Kích thước:** Chiều của vector là **768**.
 - Bạn có thể thấy điều này trong `torch.Size([1, 768])`, trong đó `1` là batch size và `768` là số chiều đặc trưng.
- Tham số tương ứng:** Có thể thấy trong ứng với tham số **hidden_size** (kích thước lớp ẩn) trong kiến trúc của mô hình **BERT Base**.
 - Mô hình BERT Base có cấu trúc: **12 layer** (41232 Attention Heads), và **768 đơn vị ẩn (hidden units)**.

2. Tại sao cần sử dụng attention_mask khi thực hiện Mean Pooling?

Mục đích chính: Để loại bỏ ảnh hưởng của các token đệm (padding tokens) đảm bảo tính chính xác của vector đại diện.

Giải thích chi tiết:

- Vấn đề Padding:** Khi xử lý theo batch, các câu ngắn phải thêm token `[PAD]` để có cùng độ dài với câu dài nhất.
- Nếu không có Mask:** Việc tính trung bình cộng (Mean) của các vector của token `[PAD]` sẽ làm vector đại diện câu bị sai lệch và "loãng" thông tin (token `[PAD]` không mang nghĩa).
- Vai trò của attention_mask:**
 - Mask chứa giá trị `1` cho token thật và `0` cho token padding.
 - Khi tính toán, chỉ tổng hợp các vector có mask là `1` và chia cho tổng số lượng token thật (thay vì chia cho tổng độ dài chuỗi).